Lctr 25: Selection Control in C and ASM

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Basic `if` construct in C and ASM.
- Basic `if-else` construct in C and ASM.
- Basic `if-else-if-else` construct in C and ASM.
- Compound `if-else` construct in C and ASM.

Demo project:

- Examples of `if`-based flow control in C and ASM.
- Exercise solutions of `if`-based flow control in C and ASM.

25.1 Introduction

We have discussed how to implement the simple forms of the `if`-based flow control in ASM by using both the conditional branch (CMP) and the conditional execution (CEX) approaches.

In this lecture, we will quickly review these basic topics using a few examples and then move on to discuss more complicated `if`-based flow control constructs in both C and ASM in a synergetic approach. All the examples are based on various activation functions used in machine learning.

25.2 C and ASM versions of a simple `if` statement

An `if` statement executes a set of statements if some conditions test true (nonzero); otherwise, it does nothing. For the simple `if` statement, the test condition is usually a simple test, such as one of the six different relational tests on two variables (numbers): `==`, `!=`, `>`, `>=`, `=<`, and `<`.

Here, we use the ReLU (rectified linear unit) function, which is a popular activation function in machine learning, to illustrate the C and ASM versions of a simple `if` statement.

$$a_{\text{relu}}(x) = \begin{cases} 0, & x < 0, \\ x, & x \geq 0. \end{cases}$$

Example 1. Implement the above ReLU function in both C and ASM assuming x is a signed integer with the following prototypes:

```c
int mp_relu_c(int x);
int mp_relu_cmp(int x);
int mp_relu_cex(int x);
```

Solution:

The C and ASM functions are shown below:

```c
int mp_relu_c(int x) {
    if (x < 0) {
        x = 0;
    }
    return x;
}
```

```
@ int mp_a_relu_cmp(int x)
@   Input:      x -> r0
@   Output:     r0
mp_a_relu_cmp:
    cmp     r0, #0
    bge     mp_a_relu_cmp_end
    ldr     r0, =0
mp_a_relu_cmp_end:
    bx      lr
```

```
@ int mp_a_relu_cex(int x)
mp_a_relu_cex:
    cmp     r0, #0
    it      lt
    ldrlt   r0, =0
    bx  lr
```

-:-

25.3 C and ASM versions of a simple `if-else` statement

Here, we use the magnified leaky ReLU (MLLU) function to illustrate C and ASM versions of a simple `if-else` statement. (Note that this is a function just constructed for illustration purposes.) We define the MLLU activation function as

$$a_{\text{mllu}}(x) = \begin{cases} x/16, & x < 0, \\ 2x, & x \geq 0. \end{cases}$$

Example 2. Implement the above MLLU function in both C and ASM assuming x is a signed integer. For C, use two opposite test conditions. For ASM, use both the CMP and CEX approaches for a chosen test condition. Use the following prototypes:

```
int mp_mulu_c1(int x);
int mp_mulu_c2(int x);
int mp_mulu_cmp(int x);
int mp_mulu_cex(int x);
```

Solution:

Below is a straightforward C function implementation:

```
int mp_a_mllu_c1(int x) {
    if (x < 0) {
        x >>= 4;
    } else {
        x <<= 1;
    }
    return x;
}
```

And here is an alternative, with the opposite test condition:

```
int mp_a_mllu_c2(int x) {
    if (x >= 0) {
        x <<= 1;
    } else {
        x >>= 4;
    }
```

```
    return x;
}
```

Here is the ASM function implementations of the second C function shown above:

```
@ int mp_a_mllu_cmp(int x)
mp_a_mllu_cmp:
    cmp     r0, #0
    blt     mp_a_mllu_cmp_else
mp_a_mllu_cmp_then:
    lsl     r0, #1
    b       mp_a_mllu_cmp_end
mp_a_mllu_cmp_else:
    asr     r0, #4
mp_a_mllu_cmp_end:
    bx      lr
```

```
@ int mp_a_mllu_cex(int x)
mp_a_mllu_cex:
    cmp     r0, #0
    ite     ge
    lslge   r0, #1
    asrlt   r0, #4
    bx      lr
```

> -:-

25.4 The `if-else-if-else` statement

Consider an example of a clipped ReLU (CRLU) activation function, which is defined as

$$a_{\text{crlu}}(x) = \begin{cases} 0, & x < 0, \\ x, & 0 \leq x < 8, \\ 8, & x \geq 8. \end{cases}$$

We also consider its derivative, which is expressed as

$$d_{\text{crlu}}(x) = \frac{d}{dx} a_{\text{crlu}}(x) = \begin{cases} 0, & x < 0 \text{ or } x \geq 8, \\ 1, & \text{otherwise.} \end{cases}$$

To implement the above CLRU activation function in C, we need to use the `if-else-if-else` construct.

Example 3. Implement the above activation function using a C function.

Solution:

Below is the C function implemented in a straightforward approach:

```c
int mp_a_crlu_c1(int x) {
    int y;
    if (x < 0) {
        y = 0;
    } else if (x < 8) {
        y = x;
    } else {
        y = 8;
    }
    return y;
}
```

A simplified implementation is given below:

```c
int mp_a_crlu_c2(int x) {
    if (x < 0) {
        x = 0;
    } else if (x >= 8) {
        x = 8;
    }
    return x;
}
```

> -:-

Example 4. Implement the above activation function using CMP and CEX, respectively, based-on the straightforward C implementation using the following prototypes:

```c
int mp_a_crlu_cmp(int x);
int mp_a_crlu_cex(int x);
```

Solution:

```
@ int mp_a_crlu_cmp(int x)
mp_a_crlu_cmp:
    cmp     r0, #0
    bge     mp_a_crlu_cmp_elseif_cmp
mp_a_crlu_cmp_then_cmp:
    ldr     r1, =0
    b       mp_a_crlu_cmp_end_cmp
mp_a_crlu_cmp_elseif_cmp:
    cmp     r0, #8
    bge     mp_a_crlu_cmp_else
    mov     r1, r0
    b       mp_a_crlu_cmp_end_cmp
mp_a_crlu_cmp_else:
    ldr     r1, =8
mp_a_crlu_cmp_end_cmp:
    mov     r0, r1
    bx      lr
```

```
@ int mp_a_crlu_cex(int x)
mp_a_crlu_cex:
    cmp     r0, #0
mp_a_crlu_cex_then_cex:
    itt     lt
    ldrlt   r1, =0
    blt     mp_a_crlu_cex_end_cex
mp_a_crlu_cex_elseif_cmp:
    cmp     r0, #8
    ite     lt
    movlt   r1, r0
    ldrge   r1, =8
mp_a_crlu_cex_end_cex:
    mov     r0, r1
    bx      lr
```

25.5 The `if-else` statement with compound test

The `if-else` statement with compound test has logic OR or AND operations in the `if` test. Here, we consider the above derivative function, reproduced below:

$$d_{\text{crlu}}(x) = \frac{d}{dx} a_{\text{crlu}}(x) = \begin{cases} 0, & x < 0 \text{ or } x \geq 8, \\ 1, & \text{otherwise.} \end{cases}$$

In the following examples, we implement the above derivative function using C and ASM functions.

Example 5. Implement the above derivative function using a C function with compound OR operation in the `if` test.

Solution:

Below is the C function:

```c
int dcrl_or_c(int x) {
    if ((x<0) || (x>=8)) {
        x = 0;
    } else {
        x = 1;
    }
    return x;
}
```

> -:-

Example 6. Implement the C function of Example 5 in ASM with the following prototype:

```
int mp_d_crlu_or_cmp(int x);
```

Solution:

```
@ int mp_d_crlu_or_cmp(int x)
@   Input:      x -> r0
@   Output:     r0
mp_d_crlu_or_cmp:
    cmp     r0, #0                  @ Check 1st condition (x < 0)
    blt     mp_d_crlu_or_cmp_then   @ Note 1.
    cmp     r0, #8
    blt     mp_d_crlu_or_cmp_else   @ If 2nd cond is false, run the else part.
mp_d_crlu_or_cmp_then:
    ldr     r0, =0
    b       mp_d_crlu_or_cmp_end
mp_d_crlu_or_cmp_else:
    ldr     r0, =1
mp_d_crlu_or_cmp_end:
    bx      lr
@ Note 1. No need to check the 2nd condition (x >= 8) if the first is true.
```

-:-

Example 7. We can easily show, by using De Morgan's law, that the above C function using OR compound condition can be easily converted to an AND compound condition shown below.

```c
int mp_d_crlu_and(int x) {
    if ((x>=0) && (x<8)) {
        x = 1;
    } else {
        x = 0;
    }
    return x;
}
```

Implement the above C function in ASM using the following prototype:

```c
int mp_d_crlu_and_cmp(int x);
```

Solution:

```
@ int mp_d_crlu_and_cmp(int x)
mp_d_crlu_and_cmp:
    cmp     r0, #0                  @ Check 1st condition (x < 0)
    blt     mp_d_crlu_and_cmp_else  @ Note A.
    cmp     r0, #8
    bge     mp_d_crlu_and_cmp_else  @ If 2nd cond is false, run the else part
mp_d_crlu_and_cmp_then:
    ldr     r0, =1
    b       mp_d_crlu_and_cmp_end
mp_d_crlu_and_cmp_else:
    ldr     r0, =0
mp_d_crlu_and_cmp_end:
    bx      lr
@ Note A. No need to check the 2nd condition (x >= 8) if the first is false.
```

> -:-

25.6 Class exercise

Exercise 1. Consider the problem of sorting an array of *two* `int16_t` elements to an ascending array. Implement it using a C function with the following prototype:

```
void mp_sort_ascend_2ele(int16_t *arr);
```

Exercise 2. Implement the C function of Exercise 1 in ASM with the following prototype:

```
void mp_sort_ascend_2ele_cmp(int16_t *arr)
```

Exercise 3. Consider the problem of copying the contents of a source array of *two* `int` elements to a destination array with the contents being sorted in the ascending order. Implement it using a C function with the following prototype:

```
void mp_copy_ascend_2ele(int *dst, int *src)
```

Exercise 4. Implement the C function of Exercise 3 in ASM with the following prototype.

```
void mp_copy_ascend_2ele_cmp(int *dst, int *src)
```

25.7 Additional reading

25.7.1 Mapping between the simple C's `if` statement and its implementation in ASM

One-to-one mapping between the C and ASM code. Note that square-bracket labels are for the C code below or to the left, and the parenthesis labels are for the ASM instructions below to to the left.

```
                                | @ int mp_relu_cmp(int x);
int mp_relu_c(int x) {          | mp_relu_cmp:
    [a]  [b]   [c]              | @      (b)
    if (x < 0) {                |        cmp     r0, #0
                                | @      (ace)
                                |        bge     relu_end        @ (NL)
         [d]                    | @      (df)
        x = 0;                  | mp_relu_then:
                                |        ldr     r0, =0
    }  [e]                      | @
    return x;  [f]              | mp_relu_end:
}                               |        bx  lr
```

One-to-one mapping between the C and ASM code.

```
                                | @ int mp_relu_cmp(int x);
int mp_mllu_c(int x) {          | mp_mllu_cmp:
    [a]  [b]    [c]             | @      (b)
    if (x >= 0) {               |        cmp     r0, #0
                                | @      (aceg)
                                |        blt     mp_mllu_else    @(NL)
          [d]                   | @      (dh)
         x <<= 1;               | mp_mllu_then:
                                |        lsl     r0, #1
    } else { [e]                | @      (eg)
                                |        b   mp_mllu_end
          [f]                   | @             (fh)
        x >>= 4;                | mp_mllu_else:
                                |        asr     r0, #4
    }  [g]                      | @
    return x; [h]               | mp_end_relu:
}                               |        bx      lr
```